

OS Skill 15

15(a) Combined Redirection

What it does: Combines stdout and stderr into one file using duplicated file descriptors and verifies the output order.

```
GNU nano 8.7.1 skill15a.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int fd;

    fd = open("combined.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("combined.txt");
        return 1;
    }

    /* Redirect stdout */
    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2 stdout");
        close(fd);
        return 1;
    }

    /* Redirect stderr to the same file descriptor */
    if (dup2(fd, STDERR_FILENO) == -1)
    {
        perror("dup2 stderr");
        close(fd);
        return 1;
    }

    close(fd);

    printf("Output: Program started.\n");
    fprintf(stderr, "Error: Test error message.\n");
    printf("Output: Program completed.\n");
}
```

```
sri_dhruthi@Dhruthi:~$ nano skill15a.c
sri_dhruthi@Dhruthi:~$ gcc skill15a.c -o skill15a
sri_dhruthi@Dhruthi:~$ ./skill15a
sri_dhruthi@Dhruthi:~$ cat combined.txt
Error: Test error message.
Output: Program started.
Output: Program completed.
```

15(b) Complex Commands – Pipes + Redirection

What it does: Demonstrates a command containing a pipe and redirection, while showing the execution order of the commands.

```
GNU nano 8.7.1 skill15b.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int pipefd[2];
    pid_t p1, p2;

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    /* First command: echo data into pipe */
    p1 = fork();

    if (p1 == 0)
    {
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("echo", "echo", "Hello Linux Pipeline", NULL);

        perror("echo");
        exit(1);
    }

    /* Second command: read from pipe and redirect to file */
    p2 = fork();

    if (p2 == 0)
    {
```

```

p2 = fork();

if (p2 == 0)
{
    FILE *file = fopen("pipeline.txt", "w");

    if (file == NULL)
    {
        perror("pipeline.txt");
        exit(1);
    }

    dup2(pipefd[0], STDIN_FILENO);
    dup2(fileno(file), STDOUT_FILENO);

    close(pipefd[0]);
    close(pipefd[1]);

    execlp("cat", "cat", NULL);

    perror("cat");
    fclose(file);
    exit(1);
}

close(pipefd[0]);
close(pipefd[1]);

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

printf("Pipeline execution completed.\n");
printf("Output saved in pipeline.txt\n");

return 0;
}

```

```

Output: Program completed.
sri_dhruthi@Dhruthi:~$ nano skill15b.c
sri_dhruthi@Dhruthi:~$ gcc skill15b.c -o skill15b
sri_dhruthi@Dhruthi:~$ ./skill15b
Pipeline execution completed.
Output saved in pipeline.txt
sri_dhruthi@Dhruthi:~$ cat pipeline.txt
Hello Linux Pipeline
sri_dhruthi@Dhruthi:~$

```